

① 把 SQL 语句转换成语法树, 如图 10.3 所示。为了使用关系代数表达式的优化法, 不妨假设内部表示是关系代数语法树, 如图 10.4 所示。

② 对语法树进行优化。利用等价变换规则 4、6 把选择 $\sigma_{SC.Cno='81003'}$ 移到叶端, 则图 10.4 所示的语法树便转换成图 10.5 所示优化后的语法树。这就是例 10.3 中 Q_3 的语法树表示。前面已经分析了 Q_3 比 Q_1 、 Q_2 的查询效率要高得多。

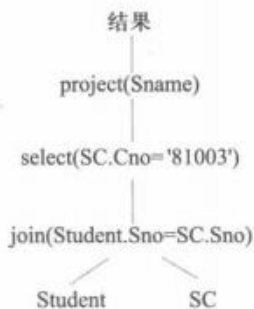


图 10.3 语法树

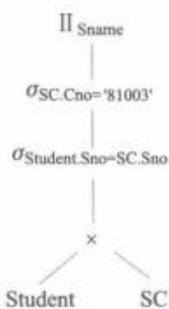


图 10.4 关系代数语法树

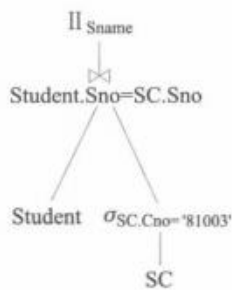


图 10.5 优化后的语法树

10.4 物理优化

代数优化改变查询语句中操作的次序和组合, 但不涉及底层的存取路径。10.1.2 小节已讲解了对每一种操作有多种执行这个操作的算法, 有多条存取路径, 因此对于一个查询语句有多种存取方案, 它们的执行效率不同, 有的会相差很大。因此, 仅仅进行代数优化是不够的。物理优化就是要选择高效合理的操作算法或存取路径, 求得优化的查询计划, 最终达到查询优化的目标。

选择的方法可以是:

① 基于启发式规则的优化。启发式规则是指那些在大多数情况下都适用, 但在不是每种情况下都是最好的规则。

② 基于代价估算的优化。使用优化器估算不同执行策略的代价, 并选出具有最小代价的执行计划。

③ 两者结合的优化方法。查询优化器通常会把这两种技术结合在一起使用。因为可能的执行策略很多, 要穷尽所有的策略进行代价估算往往是不可行的, 会导致查询优化本身付出的代价大于获得的益处。为此, 常常先使用启发式规则选取若干较优的候选方案, 减少代价估算的工作量, 然后分别计算这些候选方案的执行代价, 较快地选出最终的优化方案。

10.4.1 基于启发式规则的优化

下面以选择操作和连接操作为例, 介绍常用的启发式规则。

1. 选择操作的启发式规则

对于小关系, 使用全表顺序扫描, 即使选择列上有索引。

对于大关系, 启发式规则有:

- ① 对于选择条件是“主码=值”的查询, 查询结果最多是一个元组, 可以选择主码索引扫描。一般的关系数据库管理系统会自动建立主码索引。
- ② 对于选择条件是“非主属性=值”的查询, 并且选择列上有索引, 则要估算查询结果的元组数目, 如果比例较小($<10\%$)可以使用索引扫描方法, 否则还是使用全表顺序扫描。
- ③ 对于选择条件是属性上的非等值查询或者范围查询, 并且选择列上有索引, 同样要估算查询结果的元组数目, 如果选择率 $<10\%$ 可以使用索引扫描方法, 否则还是使用全表顺序扫描。
- ④ 对于用 AND 连接的合取选择条件, 如果有涉及这些属性的组合索引, 则优先采用组合索引扫描方法; 如果某些属性上有单属性索引, 则可以用例 10.1-C4 中介绍的索引扫描方法; 否则使用全表顺序扫描。
- ⑤ 对于用 OR 连接的析取选择条件, 一般使用全表顺序扫描。

2. 连接操作的启发式规则

- ① 如果两个表都已经按照连接属性排序, 则选用排序-合并算法。
- ② 如果一个表在连接属性上有索引, 则可以选用索引连接算法。
- ③ 如果上面两个规则都不适用, 其中一个表较小, 则可以选用哈希连接算法。
- ④ 最后可以选用嵌套循环连接算法, 并选择其中较小的表, 确切地讲是占用的块数(B)较少的表, 作为外表(外循环的表)。其理由如下:

设连接表 R 与 S 占用的块数分别为 B_R 与 B_S , 连接操作使用的内存缓冲区块数为 K , 分配 $K-1$ 块给外表。如果 R 为外表, 则嵌套循环连接算法存取的块数为 $B_R + B_R B_S / (K-1)$, 显然应该选块数小的表作为外表。

上面列出了一些主要的启发式规则, 在实际的关系数据库管理系统中启发式规则要多得多。

10.4.2 基于代价估算的优化

启发式规则优化是定性的选择, 比较粗糙, 但是它实现简单而且优化本身的代价较小, 适合用于解释执行的系统。因为解释执行的系统, 其优化开销包含在查询总开销之中。在编译执行的系统中, 一次编译优化, 多次执行, 查询优化和查询执行是分开的。因此, 可以采用更精细复杂一些的基于代价估算的优化方法。

1. 统计信息

基于代价估算的优化方法要计算各种操作算法的执行代价, 它与数据库的状态密切相关。为此在数据字典中存储了优化器需要的数据库统计信息(database statistics information), 主要包括如下几个方面:

- ① 对每个基本表, 包括该表的元组总数(N)、元组长度(l)、占用的块数(B)等。
- ② 对基本表的每个列, 包括该列不同值的个数(m)、该列最大值与最小值, 该列是否已

建立索引以及是哪种索引等。根据这些统计信息,可以计算出谓词条件的选择率(f),如果不同值的分布是均匀的, $f=1/m$;如果不同值的分布不均匀,则要计算每个值的选择率, f =具有该值的元组数/ N 。

③ 对索引,例如 $B+$ 树索引,包括该索引的层数(L)、不同索引值的个数、索引的选择基数 S (有 S 个元组具有某个索引值)、索引的叶结点数(Y)等。

2. 代价估算示例

下面给出若干操作算法的执行代价估算。所用符号 N 、 B 、 m 、 f 、 L 、 Y 等的含义如前所述,其中下标表示关系名,例如 N_R 表示关系 R 的元组数, B_R 表示关系 R 占用的块数,以此类推。

(1) 全表扫描算法的代价估算

① 如果基本表大小为 B 块,全表扫描算法的代价 $\text{cost}=B$,满足条件的元组占用 $f*B$ 块。

② 如果选择条件是“码=值”,那么平均搜索代价 $\text{cost}=B/2$,且最多只有一个元组满足条件。

(2) 索引扫描算法的代价估算

① 如果选择条件是“码=值”,如例 10.1-C2,则采用该表的主码索引。若为 $B+$ 树,层数为 L ,需要存取 $B+$ 树中从根结点到叶结点 L 块,再加上基本表中该元组所在的那一块,所以 $\text{cost}=L+1$ 。

② 如果选择条件涉及非码属性,如例 10.1-C3,若为 $B+$ 树索引,选择条件是相等比较,索引的选择基数为 N/m (有 N/m 个元组满足条件)。因为满足条件的元组可能会保存在不同的块上,所以(最坏的情况) $\text{cost}=L+N/m$ 。

③ 如果比较条件是 $>$, $>=$, $<$, $<=$ 操作,假设有一半的元组满足条件,那么就要存取一半的叶结点,并通过索引访问一半的表存储块。所以, $\text{cost}=L+Y/2+B/2$ 。如果可以获得更准确的选择基数,则可以进一步修正 $Y/2$ 与 $B/2$ 。

(3) 嵌套循环连接算法的代价估算

10.4.1 小节中已经讨论过了嵌套循环连接算法的代价 $\text{cost}=B_R+B_R B_S/(K-1)$ 。

如果需要把连接结果写回磁盘,则 $\text{cost}=B_R+B_R B_S/(K-1)+(F_{RS} * N_R * N_S)/M_{RS}$ 。其中 F_{RS} 为连接选择率,表示连接结果元组数的比例; M_{RS} 是存放连接结果的块因子,表示每块中可以存放的结果元组数目。

(4) 排序-合并连接算法的代价估算

① 如果连接表已经按照连接属性排好序,则 $\text{cost}=B_R+B_S+(F_{RS} * N_R * N_S)/M_{RS}$ 。

② 如果必须对文件排序,那么还需要在代价函数中加上排序的代价。对于包含 B 个块的文件,其外排序的代价大约是 $4B$,包括按照可用内存大小分段读入文件(代价为 B),排成有序子表并写出去(代价为 B),最后将所有的有序子表归并成全局有序文件并写回磁盘(代价为 $2B$)。因此总代价为 $\text{cost}=5B_R+5B_S+(F_{RS} * N_R * N_S)/M_{RS}$ 。

③ 在内存足够大的情形下,也可以不生成全局有序文件,而是直接在有序子表上进行合并,这样可以节省 $2B$ 的代价。

上面仅仅列出了少数操作算法的代价估算示例。在实际的关系数据库管理系中代价估算公

式要多得多,也复杂得多。

前面还提到了**基于语义的查询优化**。这种技术是根据数据库的语义约束,把原来的查询转换成另一个执行效率更高的查询。本章不对这种方法进行详细讨论,只用一个简单的例子来说明它。考虑简单的选择查询:

```
SELECT * FROM Student WHERE Smajor='计算机科学与技术' AND Sbirthdate>'2030-1-1';
```

显然,用户在写出生日期值 Sbirthdate 时,误把'2000-1-1'写成了'2030-1-1'。假设数据库模式上定义了一个约束,要求学生年龄为 15—55 岁。一旦查询优化器检查到了这条约束,它就知道上面查询的结果为空,所以根本不用执行这个查询。

*10.5 查询计划的执行

查询优化完成后,关系数据库管理系统为用户查询生成了一个查询计划。该查询计划的执行可以分为自顶向下和自底向上两种方式。

在**自顶向下的执行方式**中,系统反复向查询计划顶端的操作符发出需要查询结果元组的请求,操作符收到请求后就试图计算下一个(几个)元组并返回这些元组。在计算时,如果操作符的输入缓冲区为空,它就会向其孩子操作符发送需求元组的请求……这种需求元组的请求会一直传到叶结点,启动叶操作符运行,并返回其父操作符一个(几个)元组,父操作符再计算自己的输出返回给上层操作符,直至顶端操作符。重复这一过程,直到处理完整个关系。

在**自底向上的执行方式**中,查询计划从叶结点开始执行,叶结点操作符不断产生元组并将它们放入其输出缓冲区,直到缓冲区填满为止,这时它必须等待其父操作符将元组从该缓冲区中取走才能继续执行。然后其父结点操作符开始执行,利用下层的输入元组来产生它自己的输出元组,直到其输出缓冲区满为止。这个过程不断重复,直到产生所有的输出元组。

显然,自顶向下的执行方式是一种被动的、需求驱动的执行方式,而自底向上的执行方式是一种主动的执行方式(具体例子可见二维码内容)。详细的介绍请参阅关系数据库管理系统实现的有关文献。



扩展阅读:查询
计划执行示例

本章小结

查询处理是关系数据库管理系统的核心,而查询优化技术又是查询处理的关键技术。本章仅关注查询语句,它是关系数据库管理系统语言处理中最重要、最复杂的部分。更一般的数据库语言(包括数据定义语言、数据操纵语言、数据控制语言)处理技术可参阅关系数据库管理系统实现的有关文献。

本章讲解了基于关系代数等价变换规则的优化方法、基于启发式规则的存取路径优化和基于代价估算的优化等方法,实际系统的优化方法是综合的,优化器是十分复杂的。